

Angular Moderno

22 - Construção da Interface e criação do HeaderMenu

22 — Construção da Interface Layout do e-commerce e criação do Header/Menu

Objetivo da aula: Transformar a aplicação atual em uma interface estruturada de e-commerce, criando um layout base com header fixo e navegação entre páginas, sem alterar a lógica da aplicação.





Contexto do Projeto

Situação Atual

A aplicação já possui:

- Consumo de API
- Listagem de produtos
- Carrinho com Signals
- Rotas configuradas (Home, Produtos, Carrinho)
- Angular Material instalado

Problema Atual

- Interface ainda está desorganizada
- Navegação misturada com conteúdo
- Não há layout padrão de aplicação

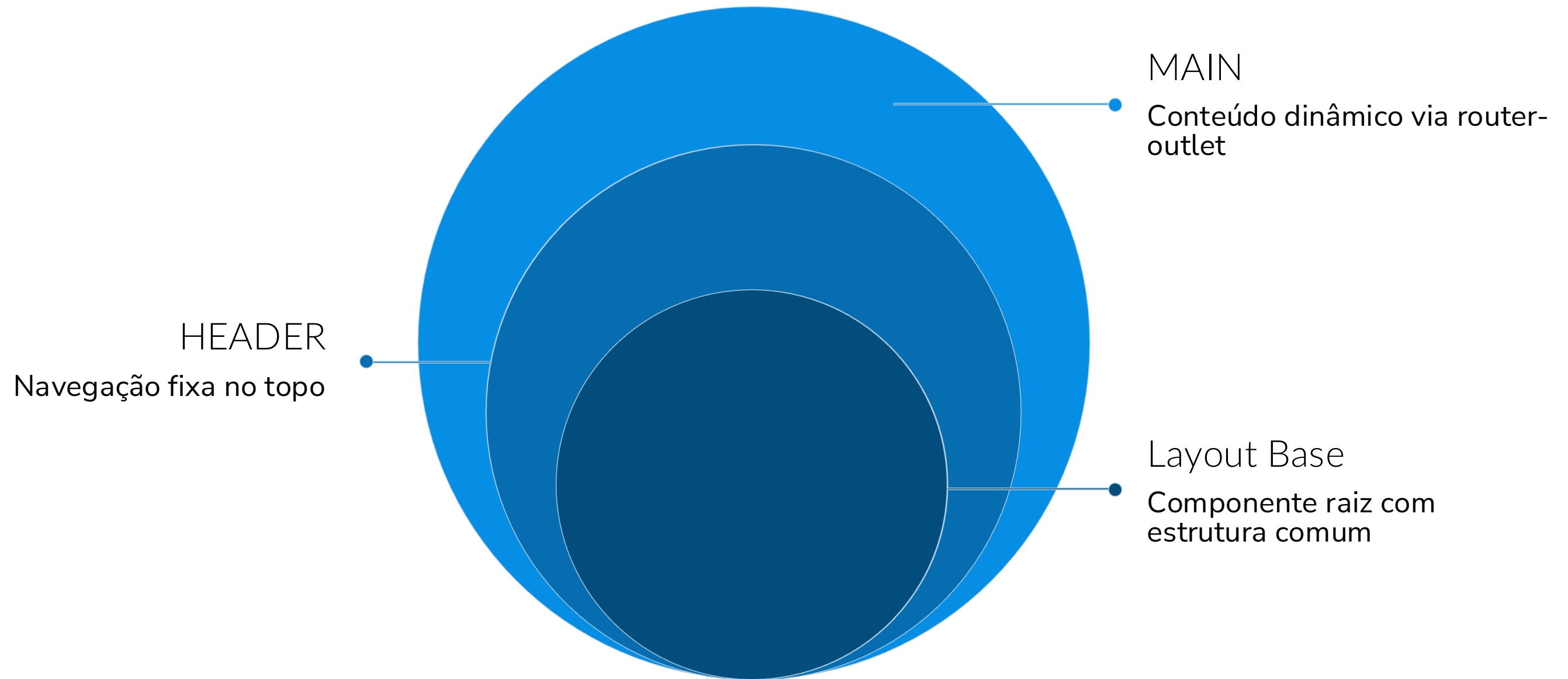
Objetivo desta Aula

Criar um layout profissional com:

- Header fixo
- Menu de navegação
- Área central controlada por rotas

Conceito Central

Toda aplicação profissional possui um layout base.



No Angular moderno, o layout fica no componente raiz e o conteúdo é renderizado via `router-outlet`.

🔗 Não vamos alterar lógica nem services. Apenas a organização **visual** da aplicação.

Etapa 0: Parar o Servidor

Ação Obrigatória

Antes de qualquer alteração estrutural, pare o servidor no terminal:

CTRL + C

Justificativa

- Evita inconsistência durante alterações
- Garante que novos arquivos sejam reconhecidos corretamente pelo Angular CLI

 Recomendado parar o servidor para evitar inconsistências

Etapa 1: Criar o Header

Objetivo: Criar o componente responsável pelo topo da aplicação.

Comando no Terminal

```
ng generate component shared/layout/header
```

Local Gerado

```
src/app/shared/layout/header/
```

Arquivos Criados

- header.ts
- header.html
- header.css

Justificativa

O Header é parte da camada de layout e deve ficar em **shared** por ser reutilizável em toda a aplicação.

Etapa 2: Importar Dependências no Header

Arquivo: header.ts

Adicionar Imports

```
import { MatToolbarModule } from '@angular/material/toolbar';  
import { MatButtonModule } from '@angular/material/button';  
import { RouterLink } from '@angular/router';
```

Atualizar Decorator

```
imports: [  
  MatToolbarModule,  
  MatButtonModule,  
  RouterLink  
],
```

MatToolbarModule

Barra superior visual do header

MatButtonModule

Botões estilizados do Material

RouterLink

Diretiva standalone que habilita navegação no template HTML. Sem ela, os botões não conseguem navegar entre rotas.

Etapa 3: Criar o Template do Header

Arquivo: `header.html`

Estrutura do Template

```
<mat-toolbar color="primary">
  <span>Minha Loja</span>
  <span class="flex-spacer"></span>
  <button mat-button routerLink="/produtos">
    Produtos
  </button>
  <button mat-button routerLink="/carrinho">
    Carrinho
  </button>
</mat-toolbar>
```

Explicação dos Elementos

- `mat-toolbar` → estrutura visual do header
- `flex spacer` → empurra os botões para a direita
- `routerLink` → navegação entre páginas sem recarregar



Etapa 4: Estilizar o Header

Arquivo: `header.css`

CSS a Adicionar

```
mat-toolbar {  
  position: sticky;  
  top: 0;  
  z-index: 1000;  
}  
  
.flex-spacer {  
  flex: 1 1 auto;  
}
```

Por que usar `sticky`?

- Mantém o header fixo no topo durante o scroll
- Melhora a experiência do usuário
- Garante visibilidade constante da navegação
- `z-index: 1000` garante que fique acima de outros elementos

Etapa 5: Importar Header no App

Arquivo: `app.ts`

1 Adicionar o Import

```
import { Header } from './shared/layout/header/header';
```

2 Adicionar no Decorator

```
imports: [RouterOutlet, RouterLink, Header],
```

 No Angular Standalone, cada componente deve declarar explicitamente o que utiliza. Sem esse import, o `app-header` não será reconhecido no HTML e causará erro de compilação.

Etapa 6: Reestruturar o App.HTML

Arquivo: `app.html` — Substituir a estrutura antiga pela nova estrutura organizada:

Nova Estrutura

```
<app-header></app-header>

<main style="padding: 20px">
  <!-- CONTROLES TEMPORÁRIOS DE LOGIN -->
  <section style="margin-bottom: 20px">
    <button (click)="login()">Login</button>
    <button (click)="logout()">Logout</button>

    <p>Status: {{ usuarioLogado() ? 'Logado' : 'Deslogado' }}</p>
  </section>

  <hr />

  <!-- CONTEÚDO DINÂMICO DAS ROTAS -->
  <router-outlet></router-outlet>
</main>
```

Explicação

- `app-header` → layout global fixo no topo
- `router-outlet` → controla o conteúdo dinâmico por rota
- Login permanece temporariamente no app até ser movido para componente próprio

Explicação da Nova Estrutura

app-header

Layout global presente em todas as páginas. Renderizado uma vez, persiste durante toda a navegação.

router-outlet

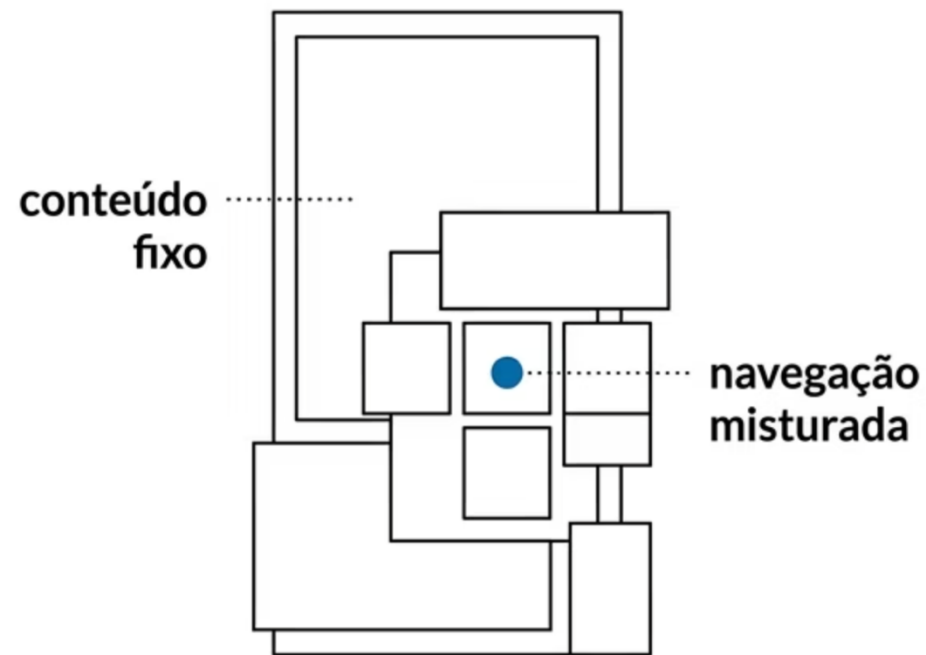
Controla o conteúdo dinâmico. Cada rota renderiza seu componente neste espaço.

Login Temporário

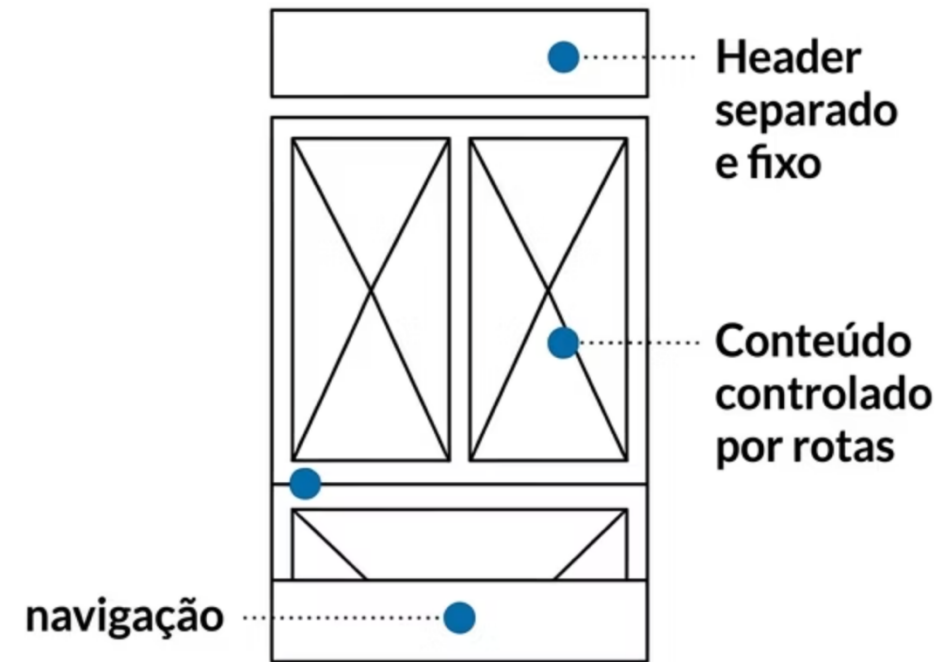
Permanece no app por enquanto. Será movido para componente dedicado em aulas futuras.

📌 Esta estrutura é a base de qualquer aplicação Angular profissional. Header global + área de conteúdo dinâmico.

Conceito Importante: Mudança Arquitetural



Conteúdo fixo e navegação misturados no app. Sem separação de responsabilidades.



Header separado, rotas controlam conteúdo, separação clara entre layout e conteúdo.

O app não decide o conteúdo. O **router** decide o conteúdo.



Validação do Funcionamento

Executar o Servidor

```
ng serve
```

O que Testar

- Header aparece no topo
- Botões Produtos e Carrinho funcionam
- Navegação troca o conteúdo
- Nenhum erro no console

Se Algo Não Funcionar

- Verificar se `RouterModule` está nos imports do `header.ts`
- Verificar se `routerLink` está correto no template HTML
- Verificar se as rotas estão configuradas no `app.routes.ts`

O Que NÃO Foi Feito (Importante)

 Esta aula é exclusivamente sobre layout e navegação. Funcionalidades adicionais serão implementadas em aulas futuras.

Estado Global

Gerenciamento de estado compartilhado entre componentes não foi implementado nesta aula.

Carrinho no Header

A integração do ícone/contador do carrinho no header ainda não foi realizada.

Services Adicionais

Nenhum novo service foi criado ou modificado. A lógica existente permanece intacta.

Motivo: Manter a separação de responsabilidades. Cada aula tem um foco claro e bem definido.

Resultado Final



Layout Estruturado

Aplicação com estrutura visual profissional e organizada



Header Fixo

Barra de navegação sempre visível durante o scroll



Navegação Funcional via Router (SPA)

Botões de menu navegam entre páginas sem recarregar



Separação Clara

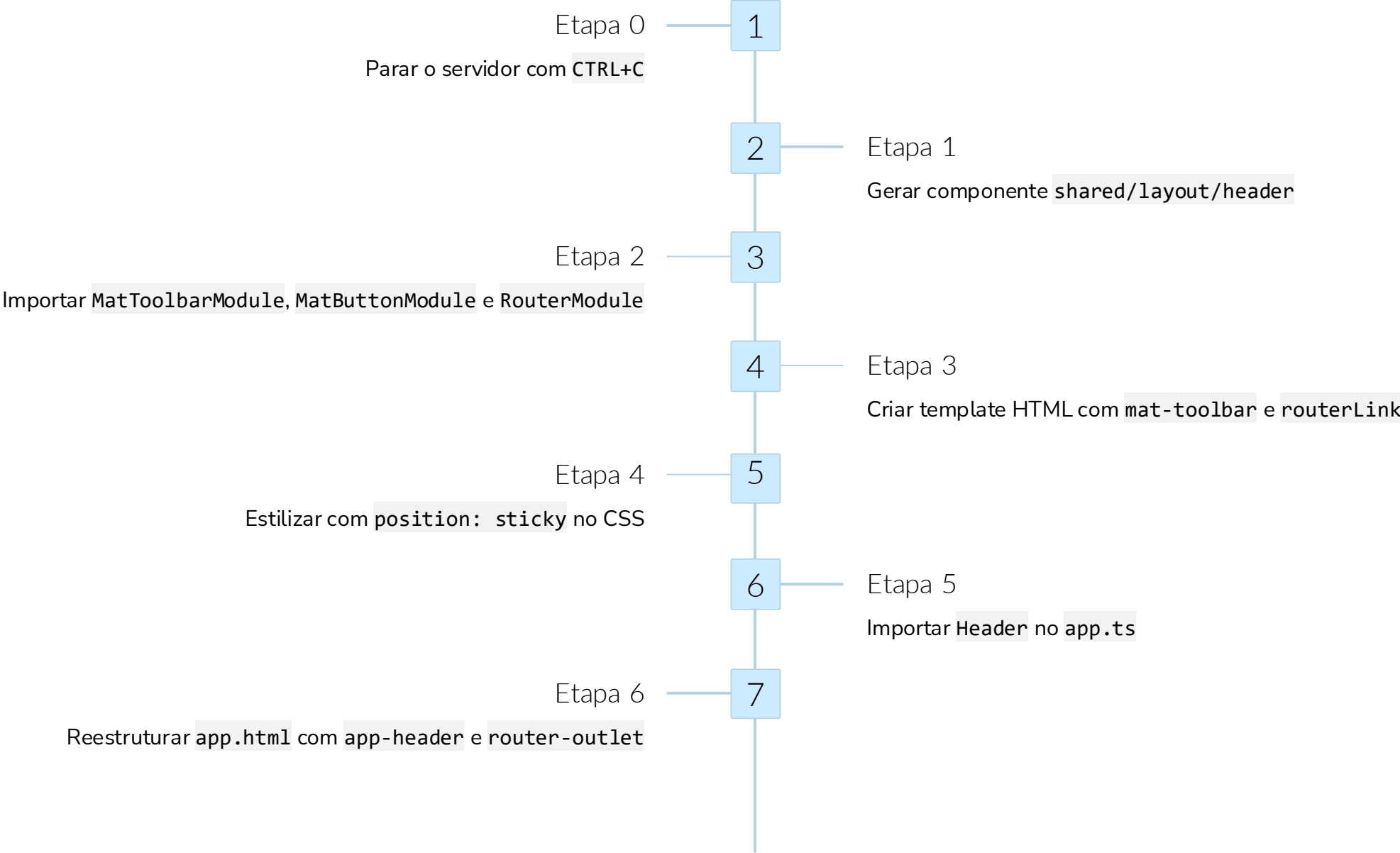
Layout e conteúdo completamente separados por responsabilidade



A base da interface está pronta para evoluções futuras!



Resumo da Aula



Dúvidas

?

